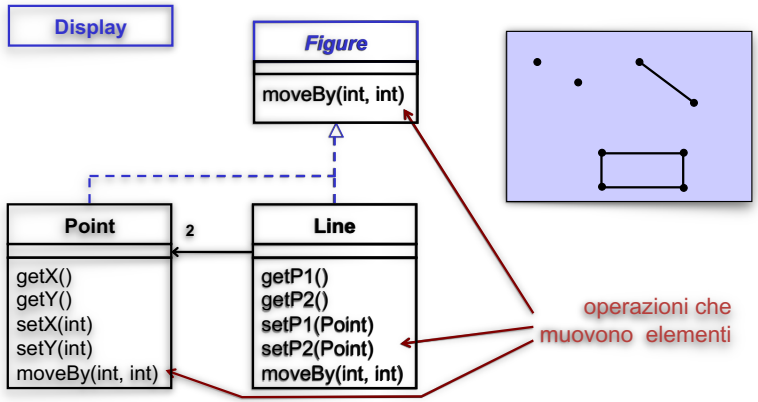
Aspect-Oriented-Programming

- Un aspetto è
  - Un modulo che permette di esprimere uno o più concern *crosscutting* in modo modulare
- Esempi tipici di aspetti e di concern crosscutting sono
  - Logging
  - Sincronizzazione
  - Sicurezza
  - Gestione della memoria

Esempio: Editor Per Figure



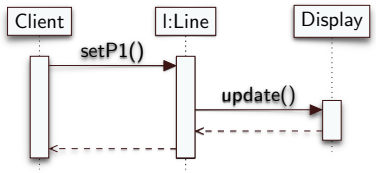

```
public interface Figure {
   public void moveBy(int dx, int dy);
}

public class Point implements Figure {
   private int x = 0, y = 0;
   public int getX() { return x; }
   public int getY() { return y; }
   public void setX(int x) { this.x = x; }
   public void setY(int y) { this.y = y; }
   public void moveBy(int dx, int dy) { /* codice */ }
}

public class Line implements Figure {
   private Point p1, p2;
   public Point getP1() { return p1; }
   public Point getP2() { return p2; }
   public void setP1(Point p1) { this.p1 = p1; }
   public void setP2(Point p2) { this.p2 = p2; }
   public void moveBy(int dx, int dy) { /* codice */ }
}
```

- Ciascun elemento che viene spostato deve essere ridisegnato, a tal fine alcuni metodi di `Point` e `Line` devono chiamare un metodo di `Display`. Tuttavia, il codice mischia due concern

Crosscutting Concern Per Display

- Senza la programmazione ad aspetti, il codice di `setP1()`, `setP2()`, `moveBy()`, deve contenere l'invocazione di `update()` della classe `Display`

```
public class Line implements Figure {
   private Point p1, p2;
   public Point getP1() { return p1; }
   public Point getP2() { return p2; }
   public void setP1(Point p1) {
      this.p1 = p1;
      Display.update();
   }
   public void setP2(Point p2) {
      this.p2 = p2;
      Display.update();
   }
   public void moveBy(int dx, int dy) {
   /* codice */
   }
}
```



- Per la versione ad oggetti, il concern "display updating" non ha un suo modulo, quindi se è necessario cambiare la visualizzazione si devono tracciare e cambiare tutte le classi chiamanti. `Line` e `Point` invece non dovrebbero saper nulla di `Display`

Aspetti

- Possono includere l'implementazione di: campi, metodi (come le classi), e costruttori (ma senza argomenti)
- Possono specificare la loro visibilità (public, etc.)
- Possono essere abstract, per essere riusati da altri aspetti, quindi avranno metodi o pointcut abstract, e non intercettano niente (poiché sono abstract)
- Possono estendere classi e aspetti abstract e implementare interfacce
- Non possono essere istanziati direttamente (non si usa il new per un aspetto), tuttavia un'unica istanza è creata automaticamente
- Non possono ereditare da aspetti non abstract

Sintassi Dei Pointcut

- I pointcut identificano i join point nel flusso del programma, possono essere dichiarati per assegnare loro un nome
- Formato di un pointcut

```
pointcut nome([param]) : designator(joinpoint);
```

- `nome` definisce un nome (stringa) per il pointcut
- `designator` permette di indicare una categoria di join point e di selezionare un join point durante l'esecuzione. Es. di designatori sono `call`, `execution`, `get`, `set`
- `joinpoint` è definito ad es. dalla signature di un metodo

Esempi Di Pointcut

- Per catturare ciascuna chiamata al metodo che ha signature

```
void Point.setX(int)
```

- Il pointcut è

```
pointcut catturaSetX() : call(void Point.setX(int));
```

- Se definiamo, per l'interfaccia Figure,

```
call(void Figure.moveBy(int, int))
```

- verrà catturata ciascuna chiamata a `moveBy` nelle classi che implementano `Figure`
- Per catturare l'istanziazione di `Point`, il pointcut è

```
call(Point.new())
```


Flusso Di Controllo

- Un pointcut è definito per catturare il controllo e far eseguire un frammento di codice (advice)

```
// pointcut che cattura il metodo setP1 di Line
pointcut myChange() : call(void Line.setP1(Point));

// advice che esegue quando il pointcut cattura il flusso
before() : myChange() {
    // corpo dell'advice
    System.out.println("prima");
}
```

Composizione Di Pointcut

- I pointcut possono essere definiti tramite l'uso di operatori boolean and, or e not (&&, ||, !)
- Per identificare e catturare entrambi i metodi `setX()` e `setY()` di `Point`

```
call(void Point.setX(int)) || call(void Point.setY(int))
```

- Per catturare metodi e tipi diversi

```
call(void Point.setX(int)) || call(void Point.setY(int))
|| call(void Line.setP1(Point))
```

- Per catturare chiamate a `setX()` di `Point`, che avvengono al di fuori di una classe (o aspetto) C

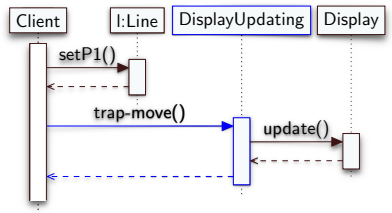
```
call(void Point.setX(int)) && !within(C)
```

- Per catturare tutti i join point possibili dentro un modulo (es. classe) che si incontrano durante l'esecuzione: `within()`

Esempio Editor Per Figure

- Con la programmazione ad aspetti, definiamo un pointcut che cattura le chiamate ai metodi `setP1()`, `setP2()`, `moveBy()` ed implementiamo un advice che chiama `update()` di `Display`
- Per il pointcut avremo l'elenco di chiamate, come per es. `call(void Line.setP1(Point))`

```
public aspect DisplayUpdating {
  pointcut move():
    call(void Figure.moveBy(int, int))||
    call(void Line.setP1(Point))      ||
    call(void Line.setP2(Point))      ||
    call(void Point.setX(int))        ||
    call(void Point.setY(int));

  after() : move() {
    Display.update();
  }
}
```



- Le chiamate di `update()` su `Display` <u>non</u> sono sparse in vari metodi delle classi `Line` e `Point`

Pointcut `target`

- I pointcut possono fornire valori del contesto
- Il pointcut `target()` fornisce il riferimento all'oggetto su cui è invocato il metodo catturato dal pointcut concatenato con &&
- Il valore del riferimento servirà agli advice

```
pointcut move(Figure fi) :
   target(fi) && call(void Figure.moveBy(int, int));

after(Figure f) : move(f) {
  // f è di tipo Figure, istanza su cui è stato chiamato moveBy
}
```


Soluzione Aspect-Oriented

- Soluzione per l'esempio dell'editor di figure in cui si deve aggiornare la visualizzazione delle figure al variare della loro posizione

```
public aspect DisplayUpdating {
   pointcut move(Figure f):
      target(f) &&
      (call(void Figure.moveBy(int, int)) ||
       call(void Line.setP1(Point))       ||
       call(void Line.setP2(Point))       ||
       call(void Point.setX(int))         ||
       call(void Point.setY(int)));

   after(Figure fe) : move(fe) {
      Display.update(fe);
   }
}
```

- Il concern è descritto in un singolo modulo, si ha quindi modularità
- Tutti i cambiamenti necessari per l'aggiornamento delle visualizzazione si possono fare in un singolo aspetto

Pointcut `args`

- Il pointcut `args()` permette di conoscere i valori degli argomenti passati nella chiamata di metodo catturato

```
pointcut move(int x, int y) :
    args(x, y) && call(void Figure.moveBy(int, int));

after(int x, int y) : move(x, y) {
    // x e y sono i valori passati a moveBy
}
```


Pointcut `this`

- Sia C una classe, `this(C)` permette di catturare tutti i join point dove il riferimento `this` del linguaggio ad oggetti punta ad un'istanza di C
- Differisce da `within()` poiché
  - `this(C)` cattura i join point durante l'esecuzione delle istanze di C o di una sua sottoclasse
  - `this(C)` non cattura l'esecuzione all'interno di classi annidate (nested)
  - `this(C)` non cattura metodi static
- Altro uso: `this(o)`
  - L'argomento specificato (o) è una variabile, che è dichiarata nella parte a sinistra del pointcut
  - Fornisce il riferimento dell'oggetto su cui avviene la cattura dell'esecuzione
- Si comporta analogamente a `target()`, ed esattamente come `target()` se il pointcut è di tipo `execution`

Advice

- Implementano azioni da eseguire ai join point catturati. Tipi di advice: `before`, `after`, `around`
- Prima di procedere al join point: `before()`
- Dopo il ritorno dal join point (ritorno in qualunque modo): `after()`
  - Per catturare quando l'esecuzione ha completato: `after() returning`
  - Per catturare il valore restituito: `after() returning(Type o)`
  - Per catturare quando è stata lanciata un'eccezione: `after() throwing`
  - Per catturare l'eccezione lanciata: `after() throwing(Throwable e)`
- Al posto del join point, ovvero all'arrivo al join point, si prende il controllo e dentro l'advice si decide se continuare l'esecuzione: `around()`
  - Per continuare l'esecuzione di ciò che è catturato: `proceed()`
  - Deve avere un valore di ritorno (poiché può sostituire l'esecuzione)


```
public aspect LoggerAsp {

   private Logger l = new Logger();

   pointcut logGet() : call(double Product.getPrice());

   after() returning(double r) : logGet() {
      l.writeLog("Price returned by getPrice");
      System.out.println("Price returned "+r);
   }

   after() throwing(Throwable e) : logGet() {
      System.out.println("Exception "+e);
   }
}
```

Advice around

```
public aspect Check {
  pointcut checkValore(int v) : // definizione pointcut
    args(v)                       &&
    (call(void Point.setX(int))  ||
     call(void Point.setY(int)));

  void around(int v) :           // advice
                checkValore(v) {  // pointcut collegato
    if (v < 9) v = 10;           // corpo dell'advice
    proceed(v);                  // esegue metodo catturato
    // do something
   }
}
```

- Tramite l'advice mostrato sopra, si legge il valore del parametro passato durante la chiamata di metodo e si cambia tale valore se non soddisfa certe condizioni

Advice around

- Quando le chiamate intercettate hanno un valore di ritorno, esso deve essere passato con return dall'advice around

```
pointcut logGet() : call(double Product.getPrice());

double around() : logGet() {
    // do something
    return proceed();
}

double around() : logGet() {
    // do something
    double x = proceed();
    // do something
    return x;
}
```


Pointcut Con Wildcard

- Per catturare join point con caratteristiche comuni si usano le wildcard
- Per indicare qualunque metodo `println()` di `PrintStream` che ritorna un `void` e prende un argomento di qualunque tipo

```
void java.io.PrintStream.println(*)
```

- Per indicare qualunque metodo `public` di `Figure`

```
public * Figure.*(..)
```

- Per indicare qualunque metodo di `Line` il cui nome inizia con `set`

```
public * Line.set*(..)
```

- `*` indica un numero qualsiasi di caratteri escluso il punto
- `..` Indica un num. qualsiasi di caratteri incluso un qualunque num. di punti
- `+` indica qualunque sottoclasse o sottointerfaccia di un tipo (es. `Figure+`)

Esempi Di Uso Di Wildcard

- Per indicare qualsiasi tipo che termina per `Product`

```
*Product
```

- Per indicare il tipo `Date` in qualsiasi package sotto il package `java`

```
java.*.Date
```

- Per indicare qualsiasi tipo all'interno del package `java` o di qualche subpackage

```
java..*
```

- Per indicare tutti i tipi all'interno del package `javax` che terminano con `Model` e tutti i loro sottotipi

```
javax..*Model+
```

- Per catturare tutte le esecuzioni di metodo

```
execution(* *.*(..))
```